

Guião de Apresentação: Sistema de Gestão de Tráfego Rodoviário (SNMP)

Oradores:

- **Orador 1 (O1):** Bernardo (1,4,7,10,13,16)
- **Orador 2 (O2):** Marcelo (2,5,8,11,14)
- **Orador 3 (O3):** Luis (3,6,9,12,15)

Slide 1 – Título O1: Boa Tarde Professor João. O nosso grupo, constituído pelo Luís, o Bernardo e o João, vai apresentar o projeto desenvolvido no âmbito da unidade curricular de Gestão de Redes: um Sistema de Gestão de Tráfego Rodoviário baseado no protocolo SNMP.

Slide 2 – O Desafio e Objetivo O2: O objetivo principal deste trabalho foi otimizar fluxos de tráfego numa zona urbana, minimizando os tempos de espera globais. O requisito fundamental era não usar protocolos proprietários, mas sim a arquitetura padrão da Internet, a *Internet-standard Network Management Framework* (INMF). Para isso, utilizámos o protocolo SNMPv2c (*Simple Network Management Protocol*) e construímos uma MIB (*Management Information Base*) experimental. A ideia central é tratar o mundo físico (estradas e semáforos) como se fossem elementos de uma rede de computadores que precisam de ser geridos.

Slide 3 – Arquitetura do Sistema O3: A nossa arquitetura segue o modelo Cliente-Servidor clássico do SNMP: À esquerda, temos a **CMC** (*Consola de Monitorização e Controlo*), que atua como o **Gestor SNMP**. É a interface do utilizador. À direita, temos o **SC** (*Sistema Central*), que funciona como o **Agente SNMP**. Dentro deste Agente, integramos dois componentes vitais:

1. O **SSFR** (*Sistema de Simulação do Fluxo Rodoviário*), que é o motor físico.
2. O **SD** (*Sistema de Decisão*), que é o cérebro que controla os semáforos. Toda a comunicação externa passa pela porta UDP 16102.

Slide 4 – Stack Tecnológica O1: Para implementar isto, utilizámos a linguagem Java. A comunicação baseia-se na biblioteca **SNMP4J** (para o gestor) e **SNMP4J-Agent** (para o agente). Optámos pelo SNMP versão 2c por ser suficiente para o protótipo, suportando o

modelo de controlo de acesso VACM. A topologia da rede rodoviária é carregada dinamicamente via ficheiro de configuração no arranque.

Slide 5 – Engenharia de Integração O2: Um dos maiores desafios foi a latência. Se o Simulador e o Decisor fossem processos separados a comunicar por rede, perderíamos o "tempo real". A nossa solução foi integrar o SSFR e o SD como *Threads* concorrentes dentro do mesmo processo Java do Agente. Assim, ambos acedem diretamente à memória RAM onde reside a MIB, eliminando o *overhead* de rede interna e permitindo uma reação imediata.

Slide 6 – A MIB Experimental O3: O coração do sistema é a nossa MIB, registada sob o OID experimental 1.3.6.1.3.2025. Estruturámo-la em três grupos principais:

1. **viaTable:** Contém o estado físico das estradas.
2. **viaDestTable:** Define a topologia e para onde os carros vão.
3. **viaControloRequest:** Um objeto escalar especial para controlo manual, que explicarei de seguida.

Slide 7 – Modelação da Via O1: Na tabela viaEntry, cada linha representa uma estrada e o seu semáforo. Guardamos atributos críticos como: o número de veículos atuais (*viaCurrentVehicles*), a cor do semáforo (*viaTrafficLightColor*), o tempo restante e o **RGT** (*Ritmo Gerador de Tráfego*), que dita quantos carros entram no sistema por minuto.

Slide 8 – Topologia e Controlo O2: Para gerir a complexidade da rede, separámos os destinos numa tabela própria (*viaDestTable*). Aqui usamos o parâmetro de distribuição (*DistributionRate*) para definir, por exemplo, que 70% do tráfego da Via A vai para o Destino B. Destaco também o nosso mecanismo de controlo "In-Band": o objeto escalar *viaControlRequest*. Funciona como uma caixa de correio (*Mailbox*). Se o Gestor quiser forçar um "Verde", escreve o ID da via neste objeto. O Decisor lê, executa e limpa o pedido.

Slide 9 – O Motor de Simulação (SSFR) O3: O Simulador corre em *background* com um passo temporal de 5 segundos. A cada ciclo, ele faz três coisas:

1. Gera novos carros com base no RGT.
2. Atualiza os contadores na MIB.

3. Move os carros se o semáforo estiver Verde ou Amarelo, respeitando a capacidade de escoamento da via e distribuindo-os pelos destinos configurados.

Slide 10 – O Algoritmo de Decisão (SD) O1: O Sistema de Decisão opera num ciclo mais lento, de 15 segundos, para evitar que os semáforos pisquem freneticamente. Utilizamos uma **Heurística Gulosa** (*Greedy*). O algoritmo varre todas as vias, identifica qual tem a fila maior (no exemplo, a Via B com 20 carros) e atribui o sinal Verde a essa via. É uma lógica simples mas eficaz para reduzir congestionamentos pontuais.

Slide 11 – Máquina de Estados O2: Para garantir a segurança rodoviária simulada, implementamos uma máquina de estados rigorosa. Nunca passamos de Verde diretamente para Vermelho. O sistema força sempre uma transição de **Amarelo fixo de 2 segundos**. Durante este período, o sistema bloqueia logicamente para garantir que a via está "limpa" antes de abrir tráfego noutra direção.

Slide 12 – Concorrência e Integridade O3: Como temos o Simulador a escrever, o Decisor a ler/escrever e o Agente SNMP a ler tudo ao mesmo tempo, o risco de corrupção de dados era alto. Resolvemos isto com **Blocos Sincronizados** (*Synchronized Blocks*) em Java e uma estratégia de **Snapshots**. Antes de processar, copiamos o estado, garantindo que ninguém altera os dados a meio de um cálculo, prevenindo as temidas *ConcurrentModificationExceptions*.

Slide 13 – Consola CMC O1: A nossa Consola (o Gestor) permite quatro operações principais:

1. Monitorização pontual.
2. Alteração dinâmica do RGT (usando *SNMP SET*).
3. Controlo Manual (o tal *override*).
4. E um modo "Live" que atualiza o dashboard continuamente, mostrando o escoamento e as filas em tempo real.

Slide 14 – Resultados (Logs) O2: Nos logs do sistema, comprovámos o funcionamento autónomo. Podemos ver o SD a avaliar a lógica e a alternar automaticamente o Verde entre a "Entrada Norte" e a "Entrada Oeste" à medida que as filas crescem e diminuem. Vê-se também explicitamente a transição de segurança: Verde -> Amarelo -> Vermelho.

Slide 15 – Validação do Ciclo de Controlo O3: Em resumo, validámos um ciclo de controlo fechado completo:

1. **Medição:** O Simulador enche as estradas e atualiza a MIB.
2. **Deteção:** O Decisor lê a MIB via memória partilhada e deteta a via crítica.
3. **Atuação:** O Decisor muda o semáforo.
4. **Feedback:** O Simulador escoar os carros e a MIB reflete a diminuição da fila, reiniciando o ciclo.

Slide 16 – Conclusão O1: Concluindo, alcançámos todos os objetivos propostos. Temos uma arquitetura Cliente-Servidor funcional, uma MIB experimental correta, e uma integração robusta entre simulação (5s) e decisão (15s). Demonstrámos que o protocolo SNMP é viável não apenas para gerir routers, mas para controlar sistemas físicos complexos como o tráfego urbano. Obrigado.

Perguntas de Preparação para a Defesa

Para o Orador 1 (Arquitetura e Conceitos)

1. **P:** *Porque optaram por integrar o simulador e o decisor no Agente como threads e não como processos externos a comunicar por SNMP traps ou sockets?*
 - o **R:** A principal razão foi a performance e a simplicidade. Processos separados introduziriam latência de rede e serialização de dados. Ao usar threads na mesma JVM (*Java Virtual Machine*), conseguimos acesso instantâneo à memória partilhada, o que é crítico para manter a simulação em "tempo real" sem desfasamentos.
2. **P:** *Porque escolheram SNMPv2c em vez de SNMPv3, dado que o v3 é mais seguro?*
 - o **R:** O foco do trabalho era a modelação da MIB e a lógica de gestão de tráfego (INMF), e não a segurança criptográfica. O SNMPv2c é mais simples de configurar e depurar (trace de pacotes) para um protótipo académico. No entanto, a biblioteca que usámos (SNMP4J) suporta v3, pelo que a migração seria direta.
3. **P:** *O que representa o OID base 1.3.6.1.3.2025 que escolheram?*
 - o **R:** Segue a estrutura padrão:
iso(1).org(3).dod(6).internet(1).experimental(3). O sufixo .2025 foi

escolhido arbitrariamente para evitar colisões com outras MIBs experimentais e identificar o ano/contexto do nosso projeto.

Para o Orador 2 (Lógica e MIB)

1. **P:** *No vosso algoritmo Greedy, o que acontece se duas vias tiverem exatamente o mesmo número de carros?*
 - **R:** O algoritmo é determinístico baseada na ordem de iteração da tabela da MIB (normalmente pelo índice). Se houver empate, a via com o índice menor (ou a primeira encontrada) ganha o Verde. Em ciclos futuros, a outra via terá acumulado mais carros e ganhará prioridade.
2. **P:** *Porque é que o passo de decisão (15s) é maior que o passo de simulação (5s)?*
 - **R:** Para garantir estabilidade (*Histerese*). Se o decisor mudasse a cada 5 segundos, passaríamos a maior parte do tempo em transições de Amarelo (2s) e perdas de arranque, reduzindo o escoamento efetivo. 15 segundos permite um período útil de Verde suficiente para escoar tráfego.
3. **P:** *Como é feito o cálculo do número de veículos a entrar com base no RGT?*
 - **R:** O RGT é definido em "veículos por minuto". No simulador, convertemos isso para o passo de 5 segundos. A fórmula é: $(RGT / 60) * 5$. Se o resultado for fracionário, usamos arredondamento ou acumulamos a fração para o ciclo seguinte para manter a precisão a longo prazo.

Para o Orador 3 (Implementação e Resultados)

1. **P:** *Como garantiram que uma leitura SNMP externa não ocorre exatamente quando o simulador está a atualizar a tabela, lendo dados inconsistentes?*
 - **R:** Utilizámos blocos synchronized no Java sobre o objeto que representa a MIB. Quando o simulador inicia a escrita, bloqueia o acesso. Se chegar um pedido SNMP nesse milissegundo, ele fica em espera até o bloqueio ser libertado. O uso de *Snapshots* (cópias locais) para o Decisor também ajuda a evitar que ele decida sobre dados que estão a mudar.
2. **P:** *O vosso sistema suporta múltiplas vias de destino para uma única via de origem? Como modelaram isso?*
 - **R:** Sim, suporta. Utilizámos a tabela viaDestTable que associa uma origem a um destino com uma probabilidade (DistributionRate). O simulador lê todas as entradas nessa tabela para a via atual e distribui os carros percentualmente (ex: 30% para a direita, 70% em frente).
3. **P:** *O que acontece se a CMC (o Gestor) falhar ou for desligada? O sistema para?*

- **R:** Não, o sistema é autónomo. O SC (Agente) contém o Simulador e o Decisor. Se a CMC for desligada, perdemos a capacidade de monitorizar ou alterar o RGT manualmente, mas os semáforos continuam a funcionar automaticamente com base na lógica do SD e nos últimos valores configurados na MIB.